

Project Final Report

Mathys Bitsch

Adrian Martinez Gao

Romain Labbe

June 1, 2026

Contents

1	Introduction	3
2	Project Evolution	3
2.1	Initial Proposal	3
2.2	Task distribution	4
3	System Overview	4
3.1	Context and Motivation	4
3.2	System Objectives	5
3.3	Proposed System	5
3.4	High-Level Pipeline	6
3.5	Results	6
3.5.1	Multi-Tracking	6
3.5.2	Fall Detection	7
4	Technical Implementation & Discussion	8
4.1	Hardware Setup	8
4.1.1	Radar hardware	8
4.1.2	Microcontroller feedback	8
4.2	Radar hardware processing	8
4.3	Software Architecture	9
4.3.1	Multiprocesses design	9
4.4	Detailed Processing Pipeline	10
4.4.1	ADC capture	10
4.4.2	CFAR detection	10
4.4.3	Beamforming	10
4.4.4	Multi-Radar Data Fusion	11
4.4.5	Output Normalization	11
4.4.6	Detection Point Extraction	11
4.4.7	Background removal	12
4.4.8	Multi-Target Tracking & Fall Detection	12
4.4.9	Hardware Signalling & Visualization	12
4.5	Dual Radar Systems	12
4.5.1	Dual Radar Fusion Framework	12
4.5.2	Radar Configurations Evaluated	13
4.5.3	Dual Radar Aperture Comparison	13

4.5.4	Parallel vs. Toed-In Geometry Analysis	14
4.6	Multi-Target Tracking	15
4.6.1	GTrack Algorithm	15
4.6.2	Track Management	16
4.6.3	Algorithmic Constraints in Multi-Target Scenarios	16
4.6.4	Technical Challenges	17
4.7	Fall Detection	17
4.7.1	Detection Strategy	17
4.7.2	Alert Generation	18
4.7.3	Fall Detection Performance	18
4.7.4	3D Beamforming	19
4.7.5	Technical Challenges	19
5	Possible extensions	19
6	Conclusion	20

1 Introduction

This final report summarizes our COM-304 semester project. We begin by outlining the evolution of the project, followed by a high-level overview of the system architecture. Next, we detail the technical implementation and discuss the challenges we faced, before concluding with potential avenues for future extensions.

2 Project Evolution

This section explains the evolution from the initial objectives in our original project proposal to the final implemented system, detailing the subsequent refinements, including which features were prioritized and which were abandoned.

2.1 Initial Proposal

The initial scope of the project primarily aimed to develop a robust multitasking radar system as a core foundation, while exploring various independent extensions. Specifically, the original proposal highlighted the following extensions:

- **Fall Detection:** A module designed to monitor sudden changes in vertical velocity and accurately detect fall events.
- **Through-Wall Detection:** Enhancing tracking algorithms to mitigate signal attenuation and multipath reflections, allowing the system to track individuals behind physical obstacles.
- **Body Shape Reconstruction:** Utilizing deep learning models, such as neural networks trained on the MPII Human Pose Dataset, to reconstruct simplified human skeletons from sparse radar point clouds.
- **Hardware-Based Feedback:** Integrating a microcontroller to control physical outputs (e.g., LEDs) to display occupant counts and tracking status as a secondary, aesthetic enhancement.

As the semester progressed, technical constraints and time limitations forced a re-assessment of these priorities. The *Body Shape Reconstruction* extension was abandoned almost immediately. This decision was driven by the high time investment required for AI training, difficulties in sourcing appropriate radar datasets, and a collective choice to maintain the project's focus on pure radar signal processing rather than deep learning.

Development initially shifted toward parallel research into both *Fall Detection* and *Through-Wall Detection*. However, toward the end of the semester, a second pivot became necessary to avoid delivering two underdeveloped features. The group chose to prioritize quality and depth over scope.

Although through-wall sensing was initially considered as a project extension, practical experiments revealed that dedicated processing was largely unnecessary for thin obstacles, as the radar could already detect and track individuals through materials such as cardboard with acceptable performance. Conversely, walls with higher attenuation significantly degraded the received signal, leaving insufficient information for robust detection. Since the achievable gains were limited and highly dependent on wall characteristics,

we chose not to pursue a dedicated through-wall module. Instead, development efforts were concentrated on optimizing radar fusion, multi-target tracking, and fall detection performance.

2.2 Task distribution

Initially we planned to divide the work in 3 : Mathys focusing on improving the multitasking baseline, Adrian working separately on fall detection and Romain on the through-wall extension. After abandoning this last extension, Romain worked with Mathys in final weeks on finishing the multitasking.

3 System Overview

This section provides a high-level overview of our system, presents its core capabilities and explains with high-level how the system works. This chapter reflects the material presented during our in-class project presentation, focusing primarily on system results and real-world applicability rather than low-level technical details. The presentation slides are available for reference in the resources section 6.

3.1 Context and Motivation

Annually, approximately one-third of adults aged 65 and older experience at least one fall. In France, this issue reached an alarming threshold with over 20,000 recorded deaths in 2024, establishing falls as the leading cause of accidental death among older adults. As the population in Europe continues to age, this public health concern is constantly growing, showing an average increase of approximately 5% each year since 2015.

Beyond the immediate physical trauma, the primary cause of mortality stems from the "long lie"—remaining immobilized on the floor for more than one hour after an accident. This delayed rescue drastically increases severe complications for older adults, including acute dehydration, muscle and tissue damage, hypothermia, and profound psychological trauma. Crucially, statistics reveal a devastating 50% mortality rate within six months following a "long lie" event, even if no severe physical injuries were sustained during the initial impact.

To mitigate this risk, automatic and continuous monitoring is essential, yet its deployment faces a major barrier regarding user privacy. Bathrooms and bedrooms represent the highest-risk areas for indoor falls; however, 80% to 90% of older adults categorically refuse the installation of traditional optical cameras in these private spaces. Furthermore, alternative wearable devices (such as panic buttons or smart bracelets) suffer from low compliance because they are frequently forgotten, uncomfortable to wear during sleep or bathing, or require manual action from an individual who may already be unconscious.

Non-optical passive sensing solutions offer a compelling path forward. Millimeter-wave radars enable fully automatic, contact-free detection. By evaluating electromagnetic wave reflections instead of visual feeds, our system operates reliably under any lighting conditions and inherently protects personal anonymity, successfully resolving the trade-off between safety and privacy.

Table 1: Technologies Comparison

Metric	Optical Cameras	Wearables (Bracelets)	Radar mmWave
Privacy Preservation	Low	High	Superior (Anonymized)
Auto-Detection	Yes	Requires Manual Action	Fully Automatic
Low Light / Bathroom	Limited	Yes	Yes (RF Sensing)
User Compliance	Low (Privacy resistance)	Medium (Often forgotten)	High (Passive sensing)

3.2 System Objectives

To effectively address the public health and privacy challenges outlined in our final presentation, the definitive objective of this project was to design, implement, and evaluate a non-intrusive, fully automated system that guarantees rapid emergency alerting while strictly respecting user privacy. Consequently, the final system was engineered to satisfy the following core technical and functional objectives:

- **Real-Time Occupant Tracking:** The system must continuously monitor the environment and track the precise spatial coordinates (X and Y positions) of individuals. This passive sensing approach removes the burden of user compliance, ensuring continuous operation without requiring active user participation while inherently preserving the occupant’s privacy.
- **Multi-User Support (Multi-Tracking):** To operate reliably in real-world environments (such as shared households or care facilities), the system must distinguish and track multiple individuals simultaneously without cross-interference or track fragmentation.
- **Robust Fall Detection:** Leveraging the spatial, velocity, and behavioral data streams collected by the mmWave sensor, the system must accurately differentiate normal daily activities (such as sitting, walking, or intentionally lying down) from actual fall events, even in challenging environmental conditions like high humidity or low lighting.
- **Alerting System:** To prevent the critical consequences of a "long lie", the system features an automated, low-latency communication layer designed to transmit instant alerts to caregivers or emergency services as soon as a fall is confirmed.

3.3 Proposed System

Our system operates by performing multi-tracking within a fixed environment using two mmWave radars. The spatial data from both sensors are dynamically fused to achieve higher localization accuracy and tracking stability.

Operating directly on top of this tracking layer, the fall detection system evaluates two primary sequential criteria to confirm a fall event:

1. **Track Disappearance:** Because the radars monitor a 2D horizontal plane and possess limited elevation resolution, a falling user transitioning to the floor causes their track to suddenly disappear from the tracked area.
2. **Velocity Threshold:** Immediately prior to track termination, the target must exhibit a significant downward velocity component (Z -axis) exceeding a pre-defined threshold.

Finally, a microcontroller interface was integrated to drive a physical buzzer and a LED upon fall confirmation, providing local feedback to simulate the behavior of a real-world emergency alerting system.

3.4 High-Level Pipeline

The processing pipeline of our system consists of the following consecutive stages:

1. **ADC Capture:** Raw data digitization (executed directly onboard the radar hardware).
2. **CFAR Detection:** Constant False Alarm Rate filtering to isolate discrete reflection points.
3. **Beamforming:** Processing point-cloud data to generate a spatial polar power intensity profile.
4. **Dual Radar Fusion:** Consolidating data matrices from both radar units into a unified coordinate space.
5. **Output Normalization:** The heatmap is power-scaled and dynamically normalized to highlight strong detections and suppress noise.
6. **Background Removal:** Dynamically filtering out static environmental reflections.
7. **Detection Point Extraction:** Extracting high-confidence target points based on peak Signal-to-Noise Ratio (SNR).
8. **Multi-Target Tracking:** State estimation and track management using the GTrack algorithm.
9. **Fall Detection:** Velocity-based anomaly evaluation.
10. **Visualization & Alert Generation:** Real-time visual display rendering.

3.5 Results

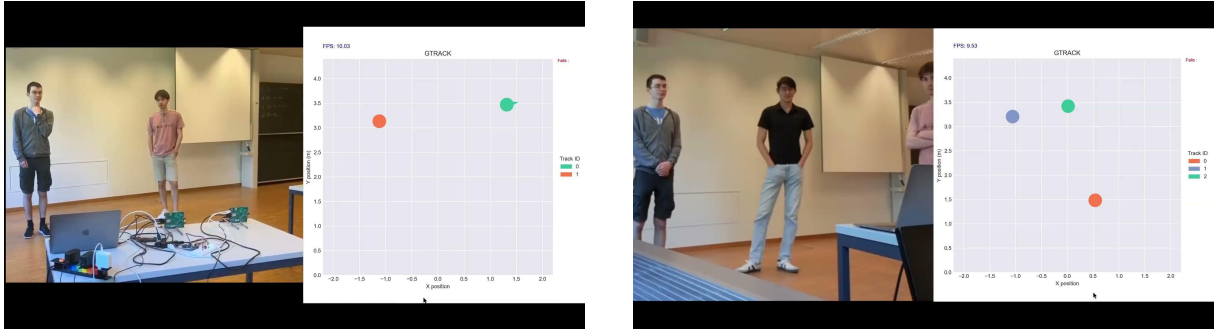
This section presents the operational capabilities achieved by the finalized system.

3.5.1 Multi-Tracking

The system demonstrates very good accuracy and reliability when tracking a single individual. It exhibits high resilience against environmental clutter and quickly initialises a stable track as soon as a subject enters the field of view. Furthermore, static objects (such as chairs) can be introduced into the environment; once background removal is recalibrated, the tracking algorithm accurately maintains the subject's path around the obstacle with high precision.

When tracking multiple individuals simultaneously, system performance degrades slightly due to spatial resolution limits. If two subjects stand in close proximity, the clustering algorithm initially attempts to merge their data points into a single track. However, once the individuals separate and are independently initialized, the tracking stabilizes as the GTrack framework explicitly manages both distinct targets. While multi-tracking for

two or three individuals remains functional, it is more susceptible to anomalies such as temporary track-merging compared to the single-user baseline.



(a) Tracking 2 persons

(b) Tracking 3 persons

Figure 1: Results of multitasking

3.5.2 Fall Detection

In the demonstration video, the first scenario depicts a person gradually transitioning from a standing position to lying on the floor in a slow, controlled manner. Despite eventually disappearing from the tracking region due to the radar's geometric bounds at ground level, no fall alarm is generated. Because the movement does not exhibit the high-velocity characteristics of an actual accident, this demonstrates the system's ability to prevent false alarms during normal daily activities.

In the second scenario, a rapid fall is executed. The system successfully isolates the sudden downward movement, correctly identifying the event as a fall and instantly triggering both the visual interface alerts and the audible hardware alarms.

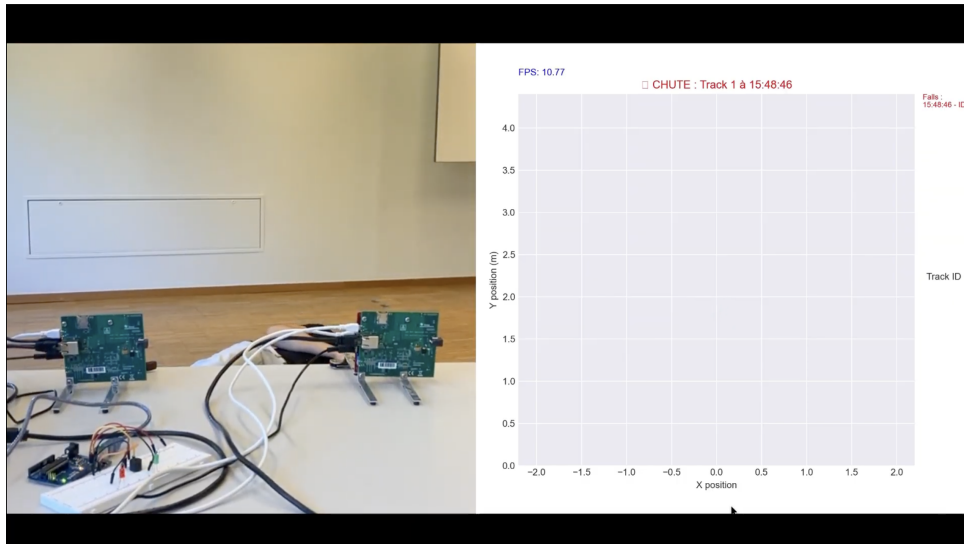


Figure 2: Project video demonstration (*Click on the image to open the Google Drive link*).

4 Technical Implementation & Discussion

This section details how the pipeline was implemented, the challenges we encountered, and the solutions we applied to resolve them.

4.1 Hardware Setup

4.1.1 Radar hardware

The system uses two TI AWR1843BOOST mmWave radar modules operating at 77 GHz. The radars are connected through DCA1000 capture boards allowing real-time acquisition of raw ADC samples.

4.1.2 Microcontroller feedback

We used an Arduino microcontroller to provide physical feedback for various states of our system: a green LED illuminates to indicate that the radar is tracking a subject; a blue LED indicates that the background removal is learning (i.e., the radar is calibrating); and a red LED combined with a buzzer activates when a fall is detected.

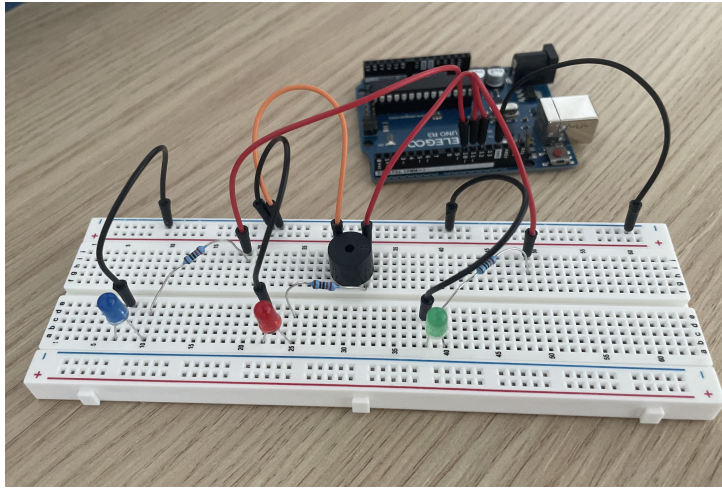


Figure 3: Arduino feedback system with LEDs and buzzer.

4.2 Radar hardware processing

The radar hardware already performs the mixing and low-pass filtering of the signal. This analog processing extracts the beat frequency before digitization, providing the raw data used for our software-based processing pipeline.

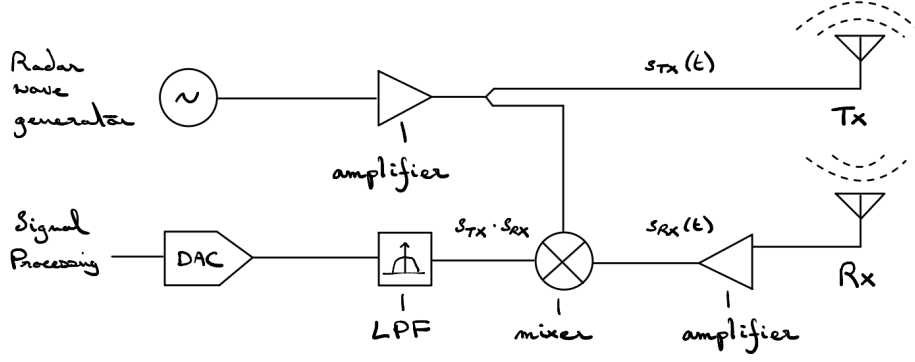


Figure 4: Radar Hardware Processing Chain

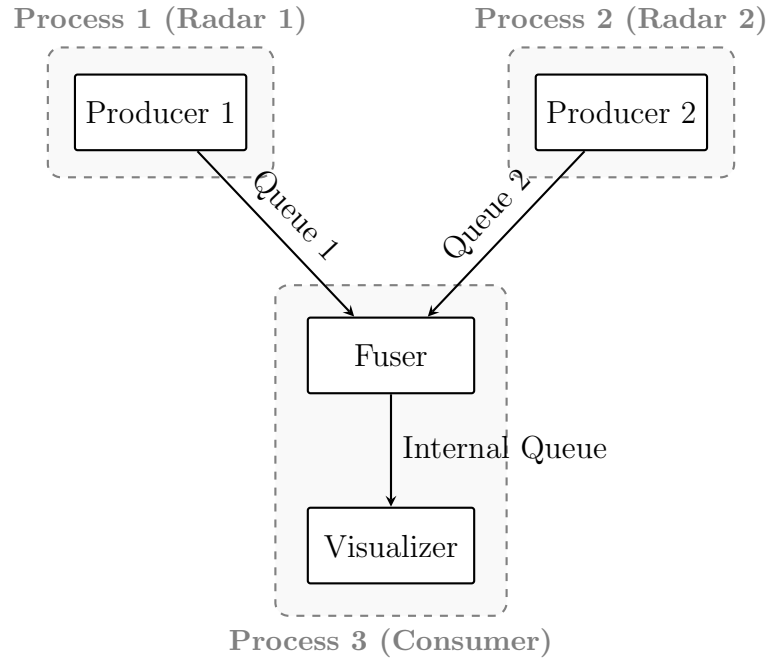
4.3 Software Architecture

4.3.1 Multiprocesses design

The software architecture relies on three independent processes:

- **Radar 1 Producer:** Handles data acquisition and processing for the first radar.
- **Radar 2 Producer:** Handles data acquisition and processing for the second radar.
- **Consumer Process:** Performs data fusion, tracking, fall detection, and visualization.

All inter-process communication relies on bounded queues with a capacity of one. A producer drops a frame if its corresponding queue is full, and the consumer always drains the queue to process only the latest data. This design guarantees real-time behavior and prevents memory growth.



4.4 Detailed Processing Pipeline

4.4.1 ADC capture

Raw complex ADC samples are streamed from the DCA1000 over Ethernet. Each frame consists of `num_tx` $\times$ `num_rx` $\times$ `num_doppler` $\times$ `num_range` complex samples. After reception the data is reorganized into a $(N_{ant}, N_{doppler}, N_{range})$ tensor where $N_{ant} = 12$ is the number of virtual antennas obtained by MIMO expansion.

A Hamming window is applied along the range dimension prior to the range FFT to reduce spectral leakage. The first 10 range bins are then zeroed out to suppress the static near-field clutter that systematically appears at very short distances.

4.4.2 CFAR detection

A 2D Cell-Averaging CFAR (CA-CFAR) is applied on the range-Doppler power map obtained by averaging the magnitude squared of the Doppler FFT across all antennas:

$$\hat{\sigma}^2(r, d) = \frac{1}{N_{train}} \sum_{(r', d') \in \mathcal{T}(r, d)} |X(r', d')|^2 \quad (1)$$

A cell is declared a detection if its power exceeds $\alpha \cdot \hat{\sigma}^2$ where α is chosen to achieve a target false-alarm rate P_{fa} :

$$\alpha = N_{train} \left(P_{fa}^{-1/N_{train}} - 1 \right) \quad (2)$$

Guard cells around the cell under test prevent the target itself from contaminating the noise estimate. The output is a boolean range-Doppler detection map used both to select bins for beamforming and as a mask for the elevation estimation.

4.4.3 Beamforming

Azimuth beamforming

For each detected bin (d, r) in the range-Doppler map, a delay-and-sum beamformer steers the 12-element virtual array across the azimuth field of view $\phi \in [-50^\circ, 50^\circ]$:

$$P(\phi, r) = \max_{(d, r) \in \text{dets}} \left| \sum_{n=0}^{N_{ant}-1} S_n(d, r) \cdot e^{j \frac{2\pi}{\lambda} x_n \sin \phi} \right| \quad (3)$$

where x_n is the azimuth position of virtual antenna n in metres. The result is a bird's-eye-view (BEV) power map of shape (N_ϕ, N_r) .

Elevation estimation

The AWR1843BOOST places TX2 one half-wavelength above TX1 in elevation. The elevation angle θ_{el} of a scatterer can therefore be estimated from the inter-TX phase difference:

$$\Delta\phi = \angle(\bar{x}_{TX2}(d, r) \cdot \bar{x}_{TX1}^*(d, r)), \quad \sin \theta_{el} = \frac{\Delta\phi}{2\pi \cdot d_{el}/\lambda} \quad (4)$$

where $d_{el} = \lambda/2$ for this board. This produces a full unmasked elevation map of shape $(N_{doppler}, N_r)$ that is forwarded to the fall detector together with the BEV map.

4.4.4 Multi-Radar Data Fusion

To expand spatial coverage and mitigate line-of-sight shadowing or occlusion effects, the spatial power maps from both radar nodes are fused within a unified Cartesian coordinate frame. The individual polar coordinates from each sensor are regularized and mapped onto a common grid using bilinear interpolation.

Let $V_1(x, y)$ and $V_2(x, y)$ denote the aligned intensity values at Cartesian coordinate (x, y) for Radar 1 and Radar 2, respectively. The fused Cartesian intensity matrix $\mathbf{Z}_{\text{cart}}$ is constructed by applying a conditional maximum-intensity heuristic:

$$\mathbf{Z}_{\text{cart}}(x, y) = \begin{cases} \max(V_1(x, y), V_2(x, y)) & \text{if } (V_1(x, y) > 0) \wedge (V_2(x, y) > 0) \\ \gamma_{\text{fusion}} \cdot \max(V_1(x, y), V_2(x, y)) & \text{if } (V_1(x, y) > 0) \oplus (V_2(x, y) > 0) \\ 0.0 & \text{otherwise} \end{cases} \quad (5)$$

where $\gamma_{\text{fusion}} = 0.8$ is an empirical attenuation scaling factor. By scaling single-sensor detections down by 20%, the fusion algorithm penalizes non-coincident returns and inherently favors joint spatial confirmations. Following matrix composition, a two-dimensional Gaussian smoothing filter is applied to $\mathbf{Z}_{\text{cart}}$ to suppress quantization artifacts and spatial noise. The smoothed field is subsequently re-interpolated back into a polar coordinate representation, denoted as $\mathbf{Z}_{\text{polar}}$.

4.4.5 Output Normalization

To compensate for range-dependent path loss and variations in target Radar Cross Sections (RCS), the fused polar heatmap undergoes dynamic amplitude scaling. The magnitude space is extracted and mapped relative to its peak frame intensity to establish a uniform baseline across varying environments:

$$\bar{\mathbf{Z}}_{\text{polar}} = \frac{|\mathbf{Z}_{\text{polar}}|}{\max(|\mathbf{Z}_{\text{polar}}|) + \epsilon} \quad (6)$$

where $\epsilon > 0$ is a small floating-point regularization parameter ensuring numerical stability during periods when no valid targets are present in the scene. This normalization bounds the heatmap intensity values strictly within the interval $[0, 1]$.

4.4.6 Detection Point Extraction

Discrete point-cloud representations are extracted from the normalized matrix $\bar{\mathbf{Z}}_{\text{polar}}$ using a localized Signal-to-Noise Ratio (SNR) thresholding mechanism. The extraction and ordering sequence is formalized as follows:

1. **Threshold Masking:** Candidate spatial indices are isolated where the normalized power meets or exceeds a predefined runtime detection threshold τ_{snr} :

$$\mathcal{I} = \{(r, \theta) \mid \bar{\mathbf{Z}}_{\text{polar}}(r, \theta) \geq \tau_{\text{snr}}\} \quad (7)$$

2. **Value Retrieval:** The discrete normalized SNR power metrics corresponding to the coordinate set $\mathcal{I}$ are compiled into an array:

$$\mathcal{V}_{\text{snr}} = \{\bar{\mathbf{Z}}_{\text{polar}}(r, \theta) \mid (r, \theta) \in \mathcal{I}\} \quad (8)$$

3. **Descending Sort Optimization:** The extracted coordinate pairs are sorted in descending order based on their associated power profiles:

$$\mathcal{I}_{\text{sorted}} = \text{sort_descending}(\mathcal{I}, \mathcal{V}_{\text{snr}}) \quad (9)$$

Sorting the point cloud ensures that subsequent tracking layers initialize and update state vectors by prioritizing dominant scattering centers over peripheral multi-path reflections.

4.4.7 Background removal

A clutter map is built by averaging the first 50 frames of the normalized fused heatmap. Once learning is complete, the clutter map is subtracted from each subsequent frame and the result is clipped to zero. This removes static reflectors such as walls, furniture and the radar mounts themselves, leaving only moving or newly appeared targets. Background removal is optional and can be disabled with a command-line flag (`-nobgrm`).

4.4.8 Multi-Target Tracking & Fall Detection

During calibration, tracking and fall detection are bypassed. Otherwise, the top 200 highest-SNR points are converted into detections for the **Gtrack** algorithm. Active tracks are then passed to the fall detector along with elevation data to identify falls based on height variations and velocity thresholds.

4.4.9 Hardware Signalling & Visualization

An Arduino interface manages physical alerts via serial bytes: `'B'/'b'` for calibration LEDs, `'G'/'g'` for active tracking LEDs, and `'R'/'r'` for critical fall alarms and buzzers. Peak projections generate 1D range/azimuth profiles, which are pushed alongside heatmaps and track data into a non-blocking queue for real-time visualization.

4.5 Dual Radar Systems

4.5.1 Dual Radar Fusion Framework

An initial engineering concern when deploying two colocated radar units was the risk of mutual radio frequency interference. However, the professor confirmed us that direct inter-sensor interference is highly improbable. Because these two FMCW radar operate on independent, unsynchronized internal clock sources, the probability of both hardware emitting identical frequency chirps at the exact same spatial instant is statistically negligible.

Throughout the project timeline, several spatial data fusion methodologies were experimentally implemented and evaluated:

- **Peak Value Retention (Maximum Fusion):** The baseline implementation fused the data arrays by selecting the maximum signal amplitude measured between the two radars at each discrete spatial coordinate. While computationally efficient and functional, this approach occasionally propagated localized noise spikes from a single radar, resulting in transient false tracks.

- **Confidence Matrix Fusion:** To improve tracking stability, a confidence-based fusion algorithm was deployed. When both radar units detect a target reflection at the exact same spatial index, the algorithm retains the peak signal maximum. Conversely, if a signature is registered by only one radar unit at that coordinate, only a fractional portion of the signal amplitude is allowed to propagate into the fused matrix. This filtering layer successfully suppresses decoupled noise spikes.
- **Weighted Averaging:** A weighted average fusion scheme was also investigated. However, this approach tended to dilute true signal peaks and did not yield acceptable data profiles; consequently, it was discarded early in the development cycle.

4.5.2 Radar Configurations Evaluated

We evaluated three physical placement configurations for the dual radar setup to optimize the target capture geometry:

1. **Parallel Configuration:** Both radars positioned on the same lateral axis facing forward, establishing a 0° relative alignment angle.
2. **Angled Configuration:** Radars placed along the same lateral axis but oriented inward toward each other at a fixed internal angle.
3. **Orthogonal Configuration:** A 90° relative placement orientation.

The implementation of the orthogonal (90°) configuration was physically constrained by the length of the available hardware interface cables, though this limitation could likely be overcome with longer cabling in future iterations. For the angled configuration, target tracking performance fell significantly short of the baseline achieved with parallel sensors. Ultimately, the parallel configuration (0° rotation, facing the same direction) yielded the highest tracking accuracy and was selected as our optimal deployment standard.



(a) Orthogonal configuration



(b) Angled configuration

Figure 5: Different radars placement

4.5.3 Dual Radar Aperture Comparison

The primary motivation for introducing a dual-radar architecture rather than relying on a single sensor was to compensate for the hardware limits of a single radar's azimuth

resolution. A single radar struggles to resolve two distinct individuals standing side-by-side at typical indoor ranges. By deploying a second radar laterally offset by a baseline of 60 cm, the tracking pipeline gains an independent angular perspective on the environment, significantly improving the system’s lateral target separation capability.

4.5.4 Parallel vs. Toed-In Geometry Analysis

Beyond evaluating single versus dual sensor configurations, we experimentally analyzed two specific structural orientations for the dual-radar baseline: a *parallel* setup where both sensors face directly forward, and a *toed-in* setup where each radar is rotated inward by approximately 20° so that their directional transmission cones converge toward the center of the monitored zone.

The toed-in geometry offers a distinct advantage when resolving occlusions: two people standing directly behind one another—aligned along the radar’s depth axis—are captured at clearly distinct azimuth angles by each respective sensor. This convergent layout creates an effective angular separation across depth-aligned targets, which is traditionally the hardest scenario for a parallel orientation to resolve.

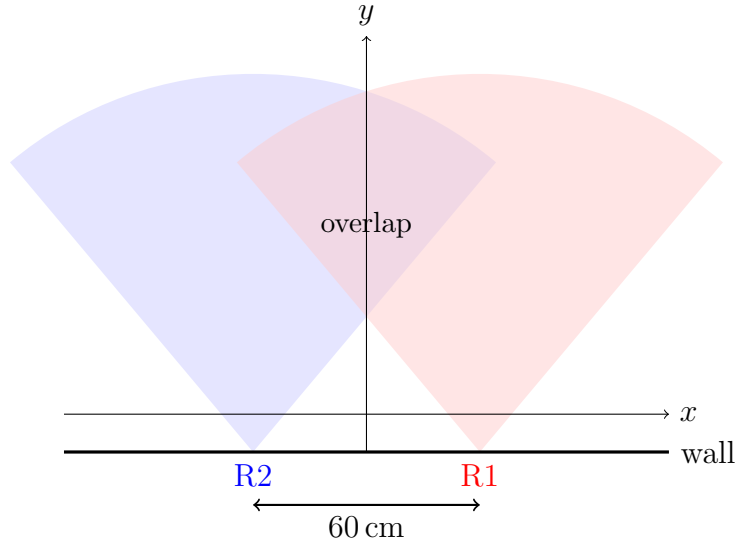


Figure 6: Coverage zones of the two parallel radars both facing forward along the y axis. The 60 cm lateral baseline creates two overlapping hemispheres, providing uniform resolution across the full monitored area.

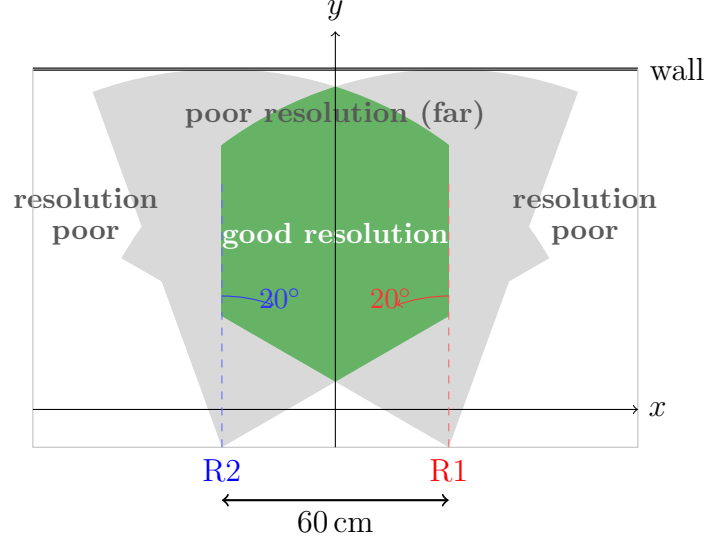


Figure 7: Toed-in configuration with both radars rotated 20° inward toward the center. White zones are completely outside both radar fields of view. Grey zones are covered by only one radar and due to the setup usually correspond to the side of one, leading to a poor resolution. The dark green zone is the overlap region where both radars contribute, yielding good resolution.

Despite these specific geometric advantages, empirical testing led to the parallel arrangement being chosen for our final system implementation. With the parallel setup, each radar unit provides its optimal angular resolution across its entire forward sensing hemisphere. Because the two sensors are laterally offset, Radar 1 monitors the right sector of the room at high resolution, Radar 2 monitors the left sector, and their patterns overlap symmetrically in the center. This yields uniformly distributed spatial resolution across the entire monitored footprint, as modeled in Figure 6.

In contrast, the toed-in geometry concentrates maximum resolution strictly within the narrow central convergence zone while severely degrading angular coverage near the lateral edges of the room (Figure 7). For a critical fall detection application, the occupant must be tracked reliably anywhere within the environment, including corners and wall perimeters. Prioritizing uniform spatial coverage over localized peak central resolution ensures that falls occurring near walls are cleanly resolved, which is vital for real-world dependability. Therefore, the parallel layout was adopted for all subsequent pipeline implementations.

4.6 Multi-Target Tracking

4.6.1 GTrack Algorithm

The GTrack module, developed by Texas Instruments (TI), implements a multi-hypothesis extended Kalman filter (EKF) tracker optimized for sparse radar point clouds. Each target is represented by a 4-dimensional state vector $\mathbf{x} = [x, y, v_x, v_y]^\top$ in Cartesian coordinates. The state transition matrix $\mathbf{F}$ models constant-velocity motion across sampling periods Δt , incorporating a small velocity decay factor of 0.97 to suppress unmodeled physical variations and prevent filter drift:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 0.97 & 0 \\ 0 & 0 & 0 & 0.97 \end{bmatrix} \quad (10)$$

Sensor measurements are natively received in polar coordinates (r, ϕ) . Data-to-track association is executed via Mahalanobis gating, where an incoming point detection is successfully associated with an ongoing filter track if its squared Mahalanobis distance to the predicted measurement falls below a configurable statistical threshold.

4.6.2 Track Management

Detections that remain unassigned after Mahalanobis gating are clustered using the DBSCAN algorithm within a normalized range-azimuth-Doppler space. The lifecycles of these clusters are systematically managed across consecutive frames:

- **Allocation:** Clusters exceeding a minimum Signal-to-Noise Ratio (SNR) threshold are dynamically allocated as *candidate* tracks.
- **Promotion:** A candidate track is promoted to an *active* state only after being continuously confirmed over several consecutive frames. This temporal filtering successfully prevents transient radar noise and clutter artifacts from spawning ghost tracks.
- **De-allocation:** Active tracks that accumulate consecutive misses due to a target exiting the scene or being occluded increment a counter. Once a specific miss threshold is reached, the track transitions back to *free*, releasing its memory slot for subsequent targets.

4.6.3 Algorithmic Constraints in Multi-Target Scenarios

While single-target tracking functions reliably within hardware limits, resolving overlapping trajectories introduces severe data association challenges. When two individuals move in close proximity or cross paths, their respective reflection point clouds merge inside a single spatial resolution cell. Because the beamformer cannot separate these signatures, the GTrack engine typically merges the tracks or absorbs one into the other. This condition persists until distinct spatial separation is re-established, occasionally resulting in identity switches or track swaps upon divergence.

These limitations scale exponentially in dense environments containing three or more moving individuals. Multiple dynamic targets frequently fall into identical angular bins, causing heavy track fragmentation.

Furthermore, higher occupant density severely constrains the spatial boundaries available during the DBSCAN clustering stage. To prevent separate people from being permanently merged into a single entity, the effective clustering neighborhood must be aggressively shrunk. This restriction significantly reduces the number of assigned reflection points per cluster, yielding highly volatile, under-sampled tracks that are exceptionally susceptible to sudden starvation, random spatial wandering, or premature de-allocation.

4.6.4 Technical Challenges

Based on these algorithmic behaviors, four primary core technical challenges govern the execution limits of the tracking pipeline:

- **Radar Resolution Bounds:** When targets share similar radial distances and angles relative to the sensors, their point clouds merge into a single spatial signature, preventing the GTrack module from distinguishing separate entities.
- **Multipath Noise and Ghost Reflections:** Radio frequency signals scatter off walls, furniture, and flooring, generating virtual targets (ghost reflections). While the dynamic background removal stage successfully suppressed static reflections, transient multipath noise from moving individuals occasionally corrupted the Mahalanobis data association, leading to short-lived tracking artifacts.
- **Physical Occlusions:** Millimeter-wave signals cannot penetrate the human body effectively. Consequently, when one individual walks directly behind another relative to the radar’s line of sight, the rear subject is completely occluded. While our dual-radar fusion architecture alleviated this by providing a second viewpoint, tight dual-axis occlusions still resulted in temporary track fragmentation.
- **Multi-Person Ambiguity in Proximity:** When multiple users interact or cross paths in close proximity, their respective state estimation covariances overlap. Without unique biometric identifiers (which standard point clouds lack), the extended Kalman filter experiences high data-association ambiguity, frequently resulting in track swaps, track starvation, or identity fragmentation upon separation.

4.7 Fall Detection

4.7.1 Detection Strategy

Fall detection is based on two complementary criteria evaluated at the moment a track disappears:

1. **Disappearance:** the track has been absent for at least T_{fall} consecutive frames (default: 20 frames at 20 fps $\approx$ 1 second). A boundary check ensures the track did not simply walk out of the monitored zone.
2. **Downward vertical velocity:** the estimated vertical speed v_z must be below a threshold $v_{z,min}$ (default: -0.8 m/s) at the time of disappearance. This discriminates genuine falls from an occupant intentionally lying down, which produces a much slower descent.

The vertical speed is estimated from the elevation angle maps produced by the two radars. For each active track the height above the floor is computed as:

$$h = h_{radar} + r \cdot \sin \theta_{el} \quad (11)$$

where h_{radar} is the radar mounting height (0.8 m in our setup) and r is the track range. A sliding window of 5 height samples is maintained per track, and consecutive differences are weighted quadratically (more recent differences receive higher weight) to produce a

robust v_z estimate that reacts quickly to the sharp downward motion of a fall without being overly sensitive to noise.

At fall decision time, the two most recent negative v_z values from the last 5 samples are extracted. If fewer than two negative values are available the missing entries are padded with zero. The final decision speed is the minimum (most negative) of these values, capturing the peak downward velocity rather than an average that would dilute the transient fall signature.

4.7.2 Alert Generation

Upon fall confirmation the system:

- Logs the event with timestamp, track ID, number of missing frames and estimated v_z .
- Triggers the red LED and buzzer on the Arduino.
- Updates the visualization title with a timestamped warning.

A special case handles multi-occupant scenarios: if at least one other track is active at the time of the fall, the red alert is suppressed and the message *"Don't worry, the person is not alone"* is displayed instead. The event is still logged internally but not appended to the public fall history. This reduces alarm fatigue in supervised environments such as care homes or shared households.

4.7.3 Fall Detection Performance

Fall detection in its current form produces a significant number of false positives. The root cause is the low accuracy of the elevation angle estimate derived from the inter-TX phase difference. The AWR1843BOOST provides only a single elevation baseline of $\lambda/2$, yielding a theoretical elevation resolution of roughly 20° . In practice, multipath reflections, phase noise, and the averaging across a sparse set of CFAR detections degrade the $\sin \theta_{el}$ estimate considerably. Small errors in $\sin \theta_{el}$ produce large errors in the estimated height $h = h_{mount} + r \cdot \sin \theta_{el}$, particularly at longer ranges, which in turn makes the vertical speed v_z unreliable as a discriminator.

As a result, the v_z threshold alone is insufficient to reliably separate falls from other rapid motions, such as crouching, quick sitting, or sharp turns, that cause a temporary track loss. While the disappearance criterion correctly flags the absence of a track, the v_z gate intended to filter out non-fall disappearances fires inconsistently due to the noisy elevation data.

However, the hardware configuration required for reliable 2D tracking imposes a fundamental constraint on fall detection accuracy. Both radars must face horizontally to cover the room plane, which restricts elevation sensing to a single $\lambda/2$ +inter-TX baseline—a geometry that yields only coarse vertical angle estimates.

The ideal solution, absent this constraint, would be to rotate one or both radars by 90° so that their high-resolution azimuth aperture is aligned with the vertical axis. Since the azimuth resolution is significantly better than the elevation resolution, a vertically oriented radar would estimate height and vertical speed with far greater accuracy, turning v_z into a reliable fall discriminator. The trade-off is that a vertically oriented radar loses its ability to locate the person horizontally. Consequently, tracking and fall detection would require

two separate radar orientations, one horizontal for localization and one vertical for height estimation, which exceeds the scope of this prototype.

4.7.4 3D Beamforming

We initially attempted to overcome these elevation resolution limits by implementing the 3D beamforming algorithm seen in course to compute instantaneous vertical velocity directly via Doppler processing. However, the high computational complexity of executing 3D spatial transforms over the point streams severely bottlenecked our processing pipeline, dropping the system frame rate from 12 fps down to an unusable 4 fps. Because a high frame rate is fundamentally required to capture rapid human falls and calculate meaningful discrete derivatives, 3D beamforming was omitted from the finalized system architecture.

A matched-filter 3D beamforming variant was subsequently explored due to its lower theoretical computational overhead. However, our handout implementation contains bugs that failed to yield stable spatial profiles. We therefore decided to rely on

4.7.5 Technical Challenges

In summary, three primary engineering bottlenecks govern the current boundaries of the fall detection framework:

- **Elevation Aperture Constraints:** The single $\lambda/2$ hardware baseline provides insufficient angular resolution, causing height calculations to degrade drastically as target range increases.
- **Velocity Discriminator Volatility:** Noisy elevation phase data propagates directly into the sliding-window derivative, creating transient velocity spikes that cause false positives during non-fall rapid motions.
- **Throughput Trade-offs:** Methods (like 3D beamforming) capable of extracting cleaner velocity data introduce severe processing overhead, conflicting with the high frame rates necessary for real-time safety monitoring.

5 Possible extensions

Several directions could extend and improve the current system.

- **Smartphone notifications:** The current alert system is limited to a local Arduino buzzer and LEDs. Integrating a network-connected notification layer, for example via SMS, push notifications, or a dedicated caregiver app, would allow remote monitoring and significantly reduce response time in real deployment scenarios.
- **Machine learning classifiers:** The current fall detection relies on hand-crafted thresholds for disappearance duration and vertical velocity. A supervised classifier trained on labelled fall and non-fall sequences could learn more complex signatures, potentially reducing both false positives from rapid crouching and false negatives from slow falls that do not trigger the v_z threshold.

- **Additional radar sensors:** A third radar oriented vertically would provide a dedicated high-resolution elevation axis, directly addressing the main limitation of our fall detection. As discussed in Section 4.7.3, the horizontal radar orientation required for 2D tracking fundamentally limits elevation accuracy. A vertical radar would resolve this without sacrificing tracking coverage.
- **Long-term activity monitoring:** Beyond fall detection, the continuous position and velocity data collected by the system could be used to build behavioural profiles over days or weeks. Anomalies in daily routines — such as prolonged immobility, unusual night-time activity, or changes in walking speed — are early indicators of cognitive or physical decline and could trigger preventive interventions before a fall even occurs.
- **Through-wall detection:** As noted in Section 2.1, our experiments showed that thin obstacles such as cardboard do not require dedicated processing. A dedicated through-wall module handling signal attenuation compensation and multipath mitigation would extend the system’s coverage to multi-room environments, allowing a single radar pair to monitor an entire apartment.

6 Conclusion

This project demonstrates a dual mmWave radar system capable of real-time occupant tracking and fall detection while preserving complete privacy. By combining standard radar signal processing (CFAR, beamforming, MIMO elevation estimation) with a robust multi-hypothesis Kalman tracker, the system achieves reliable single-person tracking and reasonable two-person tracking without any camera or wearable device.

Single-person tracking proved stable and well-suited to the target deployment context. Two-person tracking worked reasonably well. Three-person tracking under motion is an open challenge that would require higher azimuth resolution or additional sensors to address.

Fall detection shows promising results: the system correctly identifies falls in a significant portion of cases, demonstrating that the core logic — disappearance triggering, vertical speed estimation, boundary exclusion, and multi-occupancy suppression — is well-founded. However, the limited elevation accuracy of the AWR1843BOOST introduces both false positives, where rapid non-fall motions such as crouching or sudden sitting are incorrectly flagged, and false negatives, where the noisy v_z estimate fails to cross the detection threshold during a genuine fall. The modular multiprocesses architecture, real-time pipeline, and Arduino alerting system form a solid foundation on which these improvements can be built.

The clean separation between the signal processing layer, the tracking module, and the fall detection logic means that each component can be extended or replaced independently. Smartphone notification support could be added directly on top of the existing alert generation system without modifying the core pipeline. A machine learning classifier could replace the current threshold-based fall detector by learning from different fall. A vertically oriented third radar could be integrated as an additional producer process feeding elevation data into the existing fusion framework. The system is therefore not only a functional prototype but a reusable platform from which all the extensions discussed in Section 5 can be developed incrementally. We hope that our work will benefit future students in the same way that the previous team’s project helped our project development.

Project Resources

- [Code Repository](#)
- [Project presentation Slides](#)
- [Project Proposal](#)
- [Project Report](#)
- [AWR1843BOOST Documentation](#)
- [COM-304 Previous Year Project](#)